Constraint-Aware Hierarchical Deep Reinforcement Learning for Energy-Efficient Edge–Cloud Workflow Scheduling

***Abstract*—Energy-efficient scheduling of dynamically arriving edge–cloud workflows requires coordinating task placement with processor frequency: lowering frequency prolongs execution and increases server occupancy, can delay other workflows, and may reduce reliability. This paper formulates online scheduling as computation-energy minimization subject to per-workflow reliability and a probabilistic deadline-service constraint, and proposes Constraint-Aware Hierarchical Deep Reinforcement Learning (CAHDRL). A task assignment agent evaluates energy, deadline, and reliability consequences using three independent dueling Double-Q estimators, while a soft actor–critic agent selects the frequency ratio for the chosen server. Layer-specific feedback separates end-to-end assignment costs from incremental frequency-scaling costs, with independent adaptive penalty multipliers. Experiments span three workflow scales, three resource configurations, three deadline regimes, and an Alibaba-derived workload test. On the fixed scientific-workflow test sets, CAHDRL reduces mean computation energy by 10.31–57.26% relative to the lowest-energy adapted baseline in each condition. All 20 test realizations per condition meet the 90% on-time target and all workflow reliability requirements.**

***Index Terms*—Edge–cloud computing, workflow scheduling, deep reinforcement learning, DVFS, deadline constraint, reliability, adaptive penalties.**

I. Introduction

IoT applications that involve multiple processing stages can be represented as workflows of computational tasks linked by data dependencies [**?**]. Edge–cloud platforms provide heterogeneous computing resources for executing these workflows, combining processing near data sources with the capacity of remote clouds. For time-sensitive services, useful results must be delivered on time and computed correctly, making timeliness and execution reliability distinct scheduling requirements [**?**]. As workflows arrive dynamically and share computing resources, scheduling must account for both task dependencies and resource contention while limiting the energy consumed by computation.

Task placement and processor frequency jointly determine workflow execution time and energy consumption. Placement affects processing speed, data-transfer delays, and queueing, whereas dynamic voltage and frequency scaling (DVFS) reduces dynamic power at the cost of longer execution. Lower operating frequencies can also increase a task's exposure to transient faults, creating a tradeoff between energy consumption and execution reliability [**?**], [**?**]. Moreover, a task that runs longer occupies its server for an extended period and can delay tasks from other workflows sharing that server. Energy-efficient scheduling therefore requires coordinated placement and frequency decisions that account for their combined effects on workflow deadlines and reliability.

Two issues arise in this joint scheduling problem. First, server assignment is discrete, whereas frequency control is continuous and conditioned on the selected server, and the suitability of a placement changes with task dependencies and queue occupancy. A policy must coordinate the two decisions from current task and resource conditions, without access to future arrivals. Second, service requirements are defined at the workflow level, while decisions are made for individual tasks: a low average delay does not ensure that enough workflows meet their deadlines in each workload instance, and timely completion alone does not establish execution reliability. These requirements motivate a formulation that minimizes computation energy subject to per-workflow reliability and a probabilistic constraint on the fraction of workflows completed on time.

To address these issues, we propose Constraint-Aware Hierarchical Deep Reinforcement Learning (CAHDRL). A task assignment agent selects a server, and a shared frequency scaling agent determines the operating ratio for the assigned task; the agents use dueling Double Q-learning [**?**], [**?**] and soft actor–critic (SAC) [**?**], respectively. Layer-specific constraint feedback and adaptive penalties connect these task-level decisions to the workflow-level requirements. The main contributions are as follows:

1) **Hierarchical scheduling:** A discrete assignment policy and a shared, server-conditioned frequency policy coordinate placement and DVFS under dynamic workflow arrivals and resource contention.
2) **Layer-specific constraint feedback:** Separate energy and constraint estimators guide assignment, while incremental energy savings and constraint costs guide frequency control. Independent Lagrangian-inspired penalties adapt to the statistics at each level.
3) **Evaluation and component analysis:** Comparisons with three adapted baselines cover 27 scientific-workflow conditions, Alibaba-derived workloads, and workload-count and arrival-rate sensitivity. Component interventions examine placement, frequency control, and deadline-aware assignment.

II. Related Work

A. Dynamic and Learning-Based Workflow Scheduling

A recent survey by Li *et al.* [**?**] identifies resource heterogeneity, task dependencies, and system dynamics as major challenges in edge–cloud workflow scheduling. HEFT [**?**] schedules DAG tasks using upward ranks and earliest finish

times, while Arabnejad *et al.* [**?**] investigate deadline- and cost-aware scheduling for dynamically arriving cloud workflows. These studies provide foundations for dependency-aware resource allocation.

Deep reinforcement learning (DRL) has further enabled adaptive workflow scheduling. Wang *et al.* [**?**] investigate continuous workflow scheduling in heterogeneous environments, while Jayanetti *et al.* [**?**] combine hierarchical actions with actor–critic learning to balance energy and execution time in edge–cloud systems. Koslovski *et al.* [**?**] learn to select DAG scheduling policies based on workload and data-center conditions. More recently, Wen *et al.* [**?**] propose DQN-Edge, integrating dependency-aware task prioritization with separate time and energy Q-estimators and context-aware attention. Their subsequent MAES framework [**?**] adopts centralized training and decentralized execution, with virtual machines acting as collaborative scheduling agents. Xiao *et al.* [**?**] develop MCWT-AC for collaborative workflow offloading among mobile devices using actor–critic learning, while Song *et al.* [**?**] combine transformer representations with proximal policy optimization for edge DAG scheduling. These studies demonstrate the potential of learning-based scheduling, while adopting different decision structures, resource-control mechanisms, and service objectives.

B. Energy- and Reliability-Aware Workflow Scheduling

Energy-efficient workflow scheduling requires balancing resource allocation, execution speed, and reliability. Zhang *et al.* [**?**] jointly optimize workflow energy and reliability, while Xie *et al.* [**?**] investigate energy-efficient scheduling of real-time parallel applications on heterogeneous embedded systems. Oustad *et al.* [**?**] propose DIST, a distributed learning-based framework integrating task scheduling, resource allocation, and frequency control with reliability-aware mechanisms in fog computing. Ye *et al.* [**?**] investigate energy-efficient scheduling for reliability-constrained IoT workflows and subsequently coordinate reliability allocation with processor- and task-level DVFS [**?**]. These studies highlight the coupling between energy consumption and reliable execution.

Other studies incorporate explicit fault-tolerance mechanisms. Li *et al.* [**?**] combine DVFS with checkpointing and backward recovery, while Hassan *et al.* [**?**] investigate checkpoint-based recovery. Hu *et al.* [**?**] study processor consolidation and frequency scaling under deadline and reliability constraints, whereas Wu *et al.* [**?**] combine frequency adjustment with replication for periodic real-time workflows. More closely related to edge–cloud IoT workflows, Taghinezhad-Niar and Taheri [**?**] propose RECR for cost-efficient scheduling under energy, deadline, and reliability requirements. Its fault-tolerant extension, RECR-D, employs adaptive task duplication and a continuous-time Markov chain reliability model. Kouloumpris *et al.* [**?**] investigate edge–hub–cloud allocation through exact optimization and time redundancy. Unlike approaches employing task replication or fault recovery, this study considers frequency-dependent fault-free execution without additional task copies or recovery operations.

C. Hierarchical and Constrained Reinforcement Learning

Joint task allocation and frequency control involve discrete placement decisions and continuous operating frequencies. Parameterized-action reinforcement learning, such as Q-PAMDP [**?**], addresses discrete choices with associated continuous parameters, while D3PG [**?**] couples offloading proportions with frequency control. Hierarchical learning has also been investigated for workflow scheduling under hybrid-cloud privacy constraints [**?**] and global–local resource coordination with DVFS [**?**]. These approaches provide foundations for structured resource-management decisions, although their action structures and service requirements differ.

Constrained reinforcement learning provides complementary mechanisms for handling service requirements. Constrained policy optimization [**?**] addresses policy improvement under expected-cost constraints, while PID-inspired Lagrangian methods [**?**] adapt penalty multipliers in response to constraint violations. In workflow scheduling, deadline and reliability requirements are evaluated at the workflow level, whereas server assignment and frequency selection are performed for individual tasks. This mismatch between decision granularity and constraint evaluation motivates feedback mechanisms that associate task-level actions with workflow-level outcomes.

CAHDRL addresses the coupling between placement, frequency, and shared-server contention through layer-specific feedback: assignment learns end-to-end energy and constraint returns, whereas frequency control learns incremental savings and costs. Counterfactual schedule replay implements the lower-level deadline feedback, and separate adaptive penalties regulate the two decision levels.

III. System Model and Problem Formulation

A. Workflow and Task Execution Model

The principal notation is summarized in Table **??**. Let $\mathcal{W}$ be the set of workflows in a workload realization. Workflow w is a DAG of n_w tasks with task set $\mathcal{N}_w = \{1, \ldots, n_w\}$ and precedence-edge set $\mathcal{E}$. It arrives at time a_w and has a reliability requirement $R_w \in (0, 1)$. An edge $(i, j) \in \mathcal{E}$ requires task i to finish and its output to reach the server of task j before j can start; $\Phi_{\mathrm{pred}}(i)$ and $\Phi_{\mathrm{succ}}(i)$ denote the immediate predecessor and successor sets of task i. Task indices are local to each workflow, and the workflow index is omitted when the context is clear. At arrival, a workflow's DAG, task workloads, and edge data volumes are known, whereas the arrival times and attributes of future workflows are not observable. The server set $\mathcal{M} = \mathcal{M}^{\mathrm{e}} \cup \mathcal{M}^{\mathrm{c}}$ comprises edge and cloud servers: edge servers offer low communication latency but limited capacity, whereas cloud servers provide greater capacity at longer transmission delay. Each server executes at most one task at a time, non-preemptively, and supports a maximum frequency F_m and a minimum ratio $\underline{\rho}_m \in (0, 1)$, which define $F_m^{\min} = \underline{\rho}_m F_m$.

Task i has workload L_i in processor cycles and is assigned to server m_i with ratio $\underline{\rho}_{m_i} \le \rho_i \le 1$. Its execution frequency is $f_i = \rho_i F_{m_i}$ and its execution time is $ET_i = L_i / f_i$. For an edge $(i, j) \in \mathcal{E}$, D_{ij} is the transmitted data volume in bits and

TABLE I
PRINCIPAL NOTATION.

Symbol	Definition
$\mathcal{N}_w, n_w, \mathcal{E}$	Task set, task count, and precedence-edge set
$\Phi_{\mathrm{pred}}(i), \Phi_{\mathrm{succ}}(i)$	Immediate predecessor and successor sets
$a_w, T_w^{\mathrm{d}}, R_w$	Arrival time, absolute deadline, and reliability requirement
Δ_w^{d}	Workflow deadline budget
L_i, D_{ij}	Task workload (cycles) and inter-task data volume (bits)
$\mathcal{M}, m_i$	Server set and assigned server of task i
$F_m, F_m^{\min}, \underline{\rho}_m$	Maximum/minimum frequency and minimum ratio
$B_{mm'}$	Effective link bandwidth
f_i, ρ_i, ET_i	Execution frequency, scaling ratio, and execution time
c_{ij}	Communication delay
τ_{now}, A_m	Assignment time and server availability time
$RT(i), ST(i), FT(i)$	Data-ready, start, and finish times
$M_w^{(k)}, \Delta M_t$	Makespan of assigned tasks and its increment
T_w, M_w	Absolute workflow completion time and makespan
$P_m^{\mathrm{ind}}, P_m^{\mathrm{dyn}}$	Frequency-independent and dynamic power coefficients
$P_i, E_i(\rho), E_w, E_{\mathrm{sys}}$	Active power and task, workflow, and total computation energies
ρ_m^{ee}	Energy-efficient scaling ratio
$\lambda_m^{\max}, \kappa_m, \lambda_m(f)$	Maximum-frequency fault rate, sensitivity, and fault rate
$\mathcal{R}_i, \mathcal{R}_w$	Task and workflow reliabilities
$\mathrm{SR}, \Gamma, \eta, \mathrm{ESP}_\Gamma$	Success rate, rate/probability targets, and empirical success probability
$\mu_D^H, \mu_D^L, \mu_R^H, \mu_R^L$	Deadline and reliability multipliers

$B_{mm'}$ the fixed effective bandwidth between servers m and m', in bits per second. The communication delay is

$$c_{ij} = \begin{cases} \dfrac{D_{ij}}{B_{m_i m_j}}, & m_i \neq m_j, \\ 0, & m_i = m_j. \end{cases}$$

Let $RT(i)$, $ST(i)$, and $FT(i)$ denote the data-ready, start, and finish times, and let A_m denote when server m becomes available after its previously assigned tasks. At assignment time τ_{now},

$$RT(i) = \begin{cases} a_w, & \Phi_{\mathrm{pred}}(i) = \varnothing, \\ \max\limits_{j \in \Phi_{\mathrm{pred}}(i)} \{FT(j) + c_{ji}\}, & \text{otherwise}, \end{cases} \tag{1}$$

$$ST(i) = \max\{\tau_{\mathrm{now}}, RT(i), A_{m_i}\}, \tag{2}$$

$$FT(i) = ST(i) + ET_i. \tag{3}$$

Let $T_w = \max_{i \in \mathcal{N}_w} FT(i)$ denote workflow w's absolute completion time. Its makespan, measured from arrival, is $M_w = T_w - a_w$. For each workflow $w \in \mathcal{W}$, $T_w^{\mathrm{d}} > a_w$ denotes its absolute deadline and $\Delta_w^{\mathrm{d}} = T_w^{\mathrm{d}} - a_w$ its deadline budget.

B. Energy and Reliability Models

Following the active-power decomposition in [**?**], [**?**], we use a cubic frequency-dependent term [**?**], [**?**]. The active power of task i is

$$P_i = P_{m_i}^{\mathrm{ind}} + P_{m_i}^{\mathrm{dyn}} \rho_i^3,$$

where $P_m^{\mathrm{ind}} \geq 0$ is frequency-independent active power and $P_m^{\mathrm{dyn}} > 0$ is dynamic power at maximum frequency. For the assigned server m_i, task energy at a constant frequency ratio ρ is

$$E_i(\rho) = \left(P_{m_i}^{\mathrm{ind}} + P_{m_i}^{\mathrm{dyn}} \rho^3\right) \frac{L_i}{\rho F_{m_i}}.$$

The actual task energy is $E_i(\rho_i)$, workflow energy is $E_w = \sum_{i \in \mathcal{N}_w} E_i(\rho_i)$, and total computation energy is $E_{\mathrm{sys}} = \sum_{w \in \mathcal{W}} E_w$. Idle and communication energy are excluded.

Minimizing $E_i(\rho)$ over the supported frequency interval gives the energy-efficient ratio of server m:

$$\rho_m^{\mathrm{ee}} = \min\left\{1, \max\left\{\underline{\rho}_m, \left(\frac{P_m^{\mathrm{ind}}}{2P_m^{\mathrm{dyn}}}\right)^{1/3}\right\}\right\}. \tag{4}$$

This ratio depends on server power characteristics rather than task workload; meeting timing and reliability requirements may require higher frequencies.

Transient faults follow a Poisson process [**?**] with the DVFS-dependent rate of [**?**]:

$$\lambda_{m_i}(f_i) = \lambda_{m_i}^{\max} 10^{\kappa_{m_i} \frac{F_{m_i} - f_i}{F_{m_i} - F_{m_i}^{\min}}},$$

where $\lambda_m^{\max}$ is the fault rate at F_m and $\kappa_m > 0$ is the sensitivity to frequency scaling. Assuming independent task failures and no replication or recovery,

$$\mathcal{R}_i = e^{-\lambda_{m_i}(f_i) ET_i}, \qquad \mathcal{R}_w = \prod_{i \in \mathcal{N}_w} \mathcal{R}_i. \tag{5}$$

C. Optimization Problem

A workflow meets its deadline when $T_w \leq T_w^{\mathrm{d}}$. The deadline success rate over $\mathcal{W}$ is

$$\mathrm{SR} = \frac{1}{|\mathcal{W}|} \sum_{w \in \mathcal{W}} \mathbb{I}(T_w \leq T_w^{\mathrm{d}}), \tag{6}$$

where $\mathbb{I}(\cdot)$ is the indicator function, $\Gamma \in (0, 1]$ is the required success rate within a realization, and $\eta \in (0, 1]$ is the required probability of achieving it over realizations. Let π denote an online scheduling policy that selects server assignments and frequency ratios using currently available information. We minimize

$$\begin{aligned} \min_{\pi} \quad & \mathbb{E}_\pi[E_{\mathrm{sys}}] \\ \text{s.t.} \quad & \Pr_\pi(\mathrm{SR} \geq \Gamma) \geq \eta, \\ & \mathcal{R}_w \geq R_w, \qquad w \in \mathcal{W}, \\ & m_i \in \mathcal{M}, \quad \underline{\rho}_{m_i} \leq \rho_i \leq 1. \end{aligned} \tag{7}$$

The expectation and probability are over workload realizations and any policy randomization, and the precedence, communication, and non-preemptive server-capacity conditions above also apply.

IV. PROPOSED METHOD

CAHDRL separates server assignment and frequency control to use distinct actions and constraint feedback (Fig. **??**). The assignment agent receives end-to-end costs; a shared, server-conditioned frequency agent receives DVFS-induced savings and costs. Adaptive penalties regulate deadline and reliability costs during training.

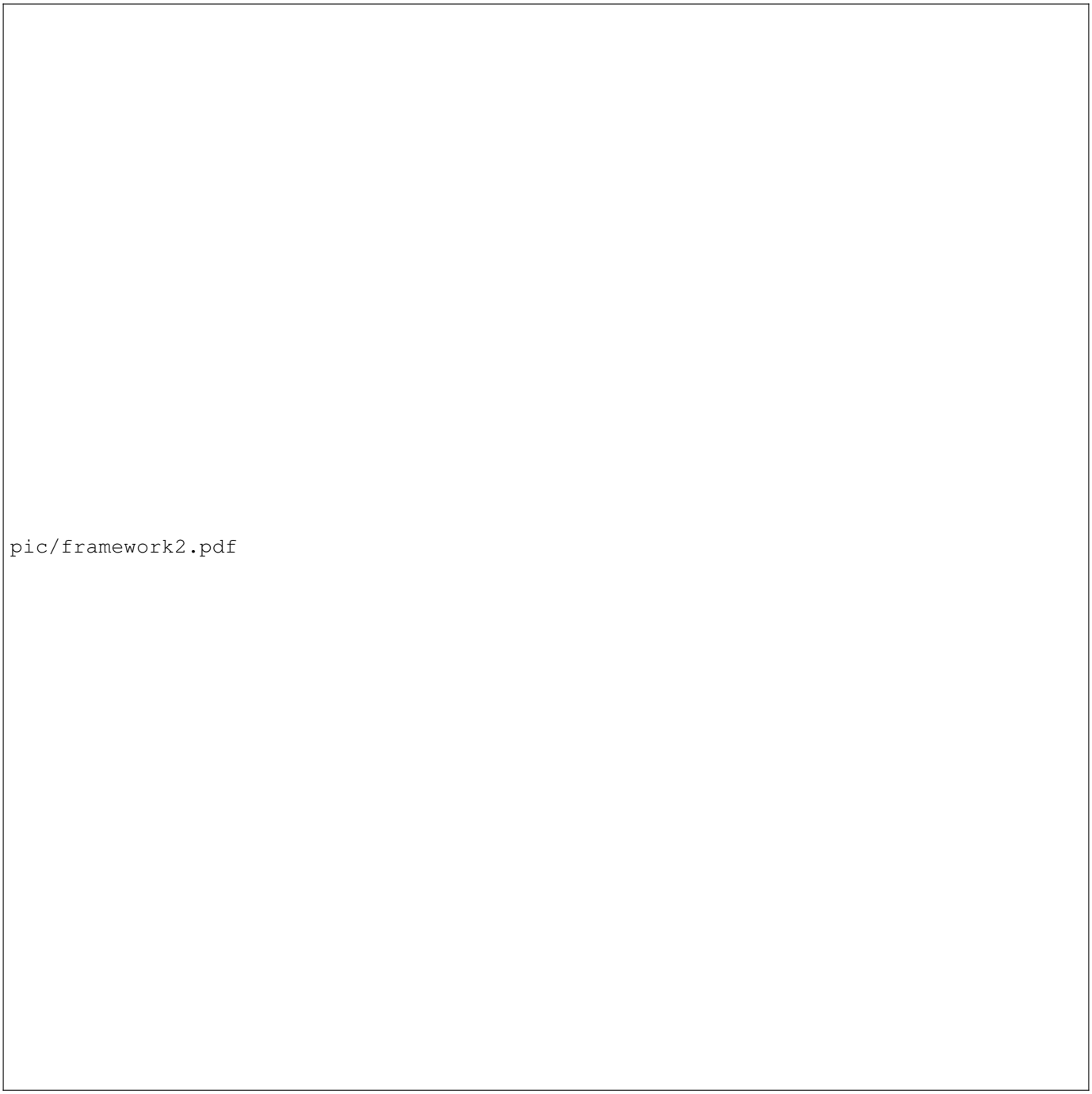


Fig. 1. Overview of CAHDRL. The task assignment agent selects a server, and the frequency scaling agent determines its DVFS ratio. Workflow outcomes provide constraint feedback for policy learning and multiplier adaptation.

A. Decision Process and Observations

At each scheduling decision, a fixed smallest-workload-first rule selects a ready task. A task becomes ready after its workflow arrives and all its predecessors finish. The assignment agent selects its server, after which the frequency scaling agent selects its operating ratio. Both decisions are fixed at assignment; a task assigned to a busy server waits for execution. The index t denotes scheduling decisions, while τ_{now} denotes physical time.

The agents act on partial observations of the scheduling state. The task graph and workloads of each workflow are known upon arrival, whereas future arrivals are unknown. Let $\boldsymbol{o}_t^H$ and $\boldsymbol{o}_t^L$ denote the assignment and frequency observations, respectively. We use $[x]_+ = \max\{0, x\}$ throughout this section.

1) Observation Space: The observations describe task requirements, resource conditions, and workflow urgency. Both agents observe the current task's workload, time remaining to its deadline, and system-wide deadline pressure. The assignment agent additionally observes task waiting time, deadline

allowance from task readiness, and candidate servers' workloads, execution and communication delays, full-frequency energy and failure probability, and recent workload and assignment history. Ready-task statistics, server occupancy, and average waiting time summarize resource contention. For the selected task–server pair, the frequency agent observes full-frequency execution time, predicted start delay, execution budget, reference frequency ratio, power coefficients, and the workflow reliability requirement.

2) Reference Frequency: The reference frequency estimates the operating ratio needed to preserve time for the remaining tasks. Let $\overline{ET}_i$ denote task i's average full-frequency execution time over servers, and $\bar{c}_{ij}$ the average communication delay over distinct server pairs. The upward rank [**?**] estimates the longest path from task i to an exit task:

$$\mathrm{rank}_u(i) = \overline{ET}_i + \max_{j\in\Phi_{\mathrm{succ}}(i)} \{\bar{c}_{ij} + \mathrm{rank}_u(j)\}.$$

The maximum term is zero for an exit task. Subtracting task i's own execution time gives $\ell_i = \mathrm{rank}_u(i) - \overline{ET}_i$, the estimated duration of the remaining path.

For candidate server m, the predicted start time $\widehat{ST}_{im}$ follows (**??**), using the current server availability and predecessor data-arrival times. After subtracting ℓ_i from the time available before the deadline, a fraction $\zeta \in (0,1)$ is reserved as a safety margin. The execution budget is therefore

$$\psi_{im} = (1-\zeta)\big(T_w^{\mathrm{d}} - \widehat{ST}_{im} - \ell_i\big).$$

For a positive budget, the reference ratio is

$$\widehat{\rho}_{im} = \min\left\{1, \max\left\{\rho_m^{\mathrm{ee}}, \frac{L_i}{F_m \psi_{im}}\right\}\right\}.$$

We set $\widehat{\rho}_{im} = 1$ when $\psi_{im} \le 0$. A smaller budget thus increases the reference ratio up to full frequency, without guaranteeing deadline feasibility.

In CAHDRL, this ratio is an observation feature, not a lower bound on the frequency action. Direct execution of the ratio defines the reference-frequency rule (RFR), evaluated in Section **??**.

3) Workflow Urgency: Workflow urgency describes the remaining processing demand relative to the time available. For this calculation, upward ranks are updated using the current scheduling state. An assigned task contributes its remaining time to completion, $[FT(i) - \tau_{\mathrm{now}}]_+$, in place of its average execution time.

For each active workflow w, the maximum below is taken over its unfinished tasks:

$$p_w = \frac{\max_i \mathrm{rank}_u(i)}{\max\{T_w^{\mathrm{d}} - \tau_{\mathrm{now}}, \epsilon_T\}},$$

where $\epsilon_T > 0$ prevents a zero denominator. The aggregate feature χ_t is obtained by summing p_w over active workflows, dividing by the number of servers, and applying $\tanh$. The maximum feature $\chi_t^{\max}$ is the largest p_w, capped at two and divided by two. Both features are zero when no workflow is active.

The assignment and frequency observations have $13|\mathcal{M}| + 12$ and 11 dimensions, respectively. Durations are normalized by the deadline budget Δ_w^{d}, while other features are scaled or bounded.

B. Hierarchical Agents

1) Task Assignment Agent: The assignment action is $m_i \in \mathcal{M}$. Three independent dueling Double-Q networks [**?**], [**?**] estimate the energy return Q_E, deadline cost Q_D, and reliability cost Q_R. Given nonnegative multipliers μ_D^H and μ_R^H, the greedy action is

$$\begin{aligned} m_i = \arg\max_{m\in\mathcal{M}} \big\{ & Q_E(\boldsymbol{o}_t^H, m) - \mu_D^H [Q_D(\boldsymbol{o}_t^H, m)]_+ \\ & - \mu_R^H Q_R(\boldsymbol{o}_t^H, m)\big\}. \end{aligned} \tag{8}$$

Training uses ε-greedy exploration.

The estimators learn their respective reward or cost returns using Double-Q targets and Huber losses. The online networks select a common next server using (**??**), and the target networks evaluate its returns. The corresponding discount factors are δ_E, δ_D, and δ_R. Deadline estimates are clipped at zero in both action selection and target bootstrapping; reliability estimates are not clipped. Learning the three returns separately allows the penalty weights to change without redefining the stored feedback.

2) Frequency Scaling Agent: A single SAC agent [**?**] serves the entire server pool. Its actor and twin critics learn from all selected task–server pairs through a shared replay buffer. The policy proposes $\rho_i^a \in [\rho_0, 1]$, where ρ_0 is a common proposal bound. The executed ratio is

$$\rho_i = \max\{\rho_i^a, \rho_{m_i}^{\mathrm{ee}}\}. \tag{9}$$

The energy-efficient ratio defined in (**??**) respects the server's hardware interval, so the executed ratio also satisfies that interval.

The twin critics estimate the entropy-regularized return of r_t^L in (**??**), with discount factor δ_L. Their targets use the smaller target-critic value, while the actor balances estimated return and policy entropy.

Critic regression uses executed ratios, whereas actor and target evaluations use sampled proposals. The two coincide only when the execution floor is inactive; floor activation therefore introduces an action-representation mismatch. The implementation is not equivalent to SAC with an explicitly transformed feasible-action parameterization.

C. Reward and Constraint Feedback

The assignment agent learns from overall scheduling outcomes, while the frequency agent receives feedback on the additional savings and costs caused by DVFS. At decision t, task i belongs to workflow w. Delayed costs are assigned to the corresponding transitions before replay insertion.

1) Energy Feedback: Using the energy model in Section **??**, the assignment reward is

$$r_t^H = \frac{\bar{E}_i - E_i(\rho_i)}{n_w \bar{E}_i}, \tag{10}$$

where $\bar{E}_i$ is the median energy for executing task i at full frequency across candidate servers. This normalized reward is a training surrogate for computation energy; evaluation uses the total energy in (**??**). For the assigned server, the energy saving and additional execution time are

$$\Delta E_i = E_i(1) - E_i(\rho_i), \qquad \Delta \mathrm{ET}_i = [ET_i - L_i/F_{m_i}]_+,$$

Let $M_w^{(k)}$ be the makespan of workflow w's assigned tasks after its k-th assignment, measured from arrival, with $M_w^{(0)} = 0$. The local delay cost measures the additional execution time relative to the timing margin before this assignment:

$$h_t = \frac{\Delta \mathrm{ET}_i}{\max\{\Delta_w^{\mathrm{d}} - M_w^{(k-1)}, \xi_h \Delta_w^{\mathrm{d}}\}}, \tag{11}$$

where $\xi_h > 0$ sets a positive denominator floor. The transformations Z_E and Z_h standardize energy savings and local delay using replay-buffer means and population standard deviations, with deviation floors and output clipping. Statistics are refreshed before each sequence of SAC updates.

2) Deadline Feedback:

a) Task assignment: Each assignment incurs a dense cost for makespan growth and server waiting:

$$d_t = \frac{\Delta M_t + \beta_q q_t / n_w}{\Delta_w^{\mathrm{d}}}, \tag{12}$$

where $\Delta M_t = M_w^{(k)} - M_w^{(k-1)}$ and $q_t = ST(i) - \max\{\tau_{\mathrm{now}}, RT(i)\}$. The makespan increments sum to M_w; the waiting term also captures contention that does not extend the makespan. Its weight $\beta_q \geq 0$ is calibrated to a prescribed share $\theta_q \in (0, 1)$ of total dense cost over an initial window and then fixed. It is zero if no server waiting is observed.

Let $\mathcal{W}_e$ denote the workflows in trajectory e. Workflow and trajectory failures are indicated by $z_w = \mathbb{I}(T_w > T_w^{\mathrm{d}})$ and $z_e = \mathbb{I}(\mathrm{SR}_e < \Gamma)$, respectively. Using the full-frequency finish estimate $\widehat{FT}_{im} = \widehat{ST}_{im} + L_i/F_m$, the assignment regret is

$$g_t = \frac{\widehat{FT}_{im_i} - \min_m \widehat{FT}_{im}}{\Delta_w^{\mathrm{d}}},$$

where the minimum is over candidate servers. The complete deadline cost is

$$c_{D,t}^H = d_t + \beta_{\mathrm{miss}}(z_w \omega_t + z_e \nu_t). \tag{13}$$

Here $\beta_{\mathrm{miss}} \geq 0$ scales failure penalties, ω_t is the decision's share of its workflow's total dense cost, and ν_t is its share of the trajectory's total assignment regret. If either total is zero, the corresponding penalty is allocated uniformly over the workflow or trajectory decisions.

b) Frequency scaling: DVFS-induced lateness is estimated by replaying completed schedules with selected workflows restored to full frequency. Server assignments, execution order on each server, communication delays, and original assignment times remain fixed. Replay reuses (**??**)–(**??**): selected tasks use execution time L_i/F_{m_i}, while other tasks retain their original execution times. Data-ready times and server availability are recomputed from replayed finish times; τ_{now} in (**??**) is the task's original assignment time. An earlier server release therefore does not advance a task still blocked by data readiness or its assignment time.

Let M_v^{all} and $M_v^{u\to 1}$ be workflow v's makespans when all workflows, or only workflow u, are restored to full frequency. With normalized lateness $\mathcal{L}_v(M) = [M/\Delta_v^{\mathrm{d}} - 1]_+$, define

$$\begin{aligned} d_v^L &= \min\{1, [\mathcal{L}_v(M_v) - \mathcal{L}_v(M_v^{\mathrm{all}})]_+\}, \\ \Delta_{uv} &= [\mathcal{L}_v(M_v) - \mathcal{L}_v(M_v^{u\to 1})]_+. \end{aligned} \tag{14}$$

The cost d_v^L measures lateness attributable to DVFS, capped at one to limit each workflow's contribution. The marginal Δ_{uv} measures the improvement for workflow v when workflow u runs at full frequency.

Costs are allocated first to source workflows according to these marginals, then to their tasks according to additional execution time:

$$G_u = \sum_v \frac{\Delta_{uv}}{\sum_{u'} \Delta_{u'v}} d_v^L, \tag{15}$$

$$c_{D,t}^L = G_w \frac{\Delta \mathrm{ET}_i}{\sum_{j \in \mathcal{N}_w} \Delta \mathrm{ET}_j}. \tag{16}$$

The workflow sums range over the current trajectory. If all marginals for an affected workflow vanish, its cost is allocated by source workflows' total additional execution times; no cost is allocated if that total is also zero. If the task-level denominator is zero, the source cost is allocated uniformly within that workflow. Each affected cost is allocated once; single-workflow marginals approximate joint effects rather than providing an exact additive decomposition.

3) Reliability Feedback: The assignment agent receives $c_{R,t}^H = d_w^R$ at workflow w's last decision and zero otherwise, where $d_w^R = [\ln R_w - \ln \mathcal{R}_w]_+$ is its final deficit.

For the frequency agent, $\widehat{\mathcal{R}}_{w,t}(f)$ is the projected workflow reliability when the current task runs at frequency f. It combines assigned-task reliabilities with a fixed per-task estimate for unscheduled tasks. Only the current task's factor changes between the two frequency choices. For $0 < R_w < 1$,

$$\begin{aligned} u_t(f) &= \frac{[\ln R_w - \ln \widehat{\mathcal{R}}_{w,t}(f)]_+}{-\ln R_w}, \\ c_{R,t}^L &= [u_t(f_i) - u_t(F_{m_i})]_+. \end{aligned}$$

Thus, the lower cost measures only the additional projected deficit caused by frequency reduction.

The frequency agent's reward combines these signals:

$$r_t^L = Z_E(\Delta E_i) - Z_h(h_t) - \mu_D^L c_{D,t}^L - \mu_R^L c_{R,t}^L. \tag{17}$$

Local delay penalizes consumption of timing margin, while the deadline cost measures additional lateness across dependent tasks and shared servers. Only energy savings and local delay are standardized.

D. Lagrangian-Inspired Constraint Handling

The four multipliers in (**??**) and (**??**) regulate deadline and reliability penalties separately at each decision level. Independent weights accommodate the different scales of end-to-end and incremental costs. These updates are inspired by Lagrangian constraint handling; they do not solve the exact dual of (**??**).

1) Deadline Multiplier Updates: For task assignment, J_H averages recent indicators $z_e = \mathbb{I}(\mathrm{SR}_e < \Gamma)$. Here Γ sets the within-trajectory success threshold, while $b_H = 1 - \eta$ is the allowable trajectory-failure probability in (**??**). For frequency scaling, each trajectory contributes the average affected-workflow cost $|\mathcal{W}_e|^{-1} \sum_{v \in \mathcal{W}_e} d_v^L$, before allocation to source decisions. The statistic J_L averages the largest prescribed fraction of these values in a recent window, with

the number selected rounded upward. Its budget $b_L \in [0,1]$ controls the upper tail of DVFS-induced lateness. Following proportional–integral (PI) adaptation [**?**], let U_j accumulate past budget deviations, initialized to the starting multiplier. Let $\tau_j > 0$ be its step size, $k_j \geq 0$ the proportional gain, $e_j = J_j - b_j$ the error, and $\bar{\mu}_D^j > 0$ the multiplier limit for $j \in \{H, L\}$:

$$U_j \leftarrow \Pi_{[0,\bar{\mu}_D^j]}(U_j + \tau_j e_j),$$
$$\mu_D^j \leftarrow \Pi_{[0,\bar{\mu}_D^j]}(U_j + k_j e_j),$$

where $\Pi_{[a,b]}$ projects onto $[a,b]$ and the second line uses the updated U_j. The accumulated term retains past deviations, while $k_j e_j$ responds to the current error. Updates begin after warm-up and once enough observations have been collected.

2) Reliability Multiplier Updates: The upper multiplier uses the final deficit d_w^R, while the lower multiplier uses the recent mean J_R of $\mathbb{I}(\mathcal{R}_w < R_w)$. For step sizes $\tau_R^H, \tau_R^L > 0$, limits $\bar{\mu}_R^H, \bar{\mu}_R^L > 0$, and budget $b_R \geq 0$,

$$\begin{aligned} \mu_R^H &\leftarrow \Pi_{[0,\bar{\mu}_R^H]}(\mu_R^H + \tau_R^H d_w^R), \\ \mu_R^L &\leftarrow \Pi_{[0,\bar{\mu}_R^L]}\left(\mu_R^L + \tau_R^L (J_R - b_R)\right). \end{aligned} \tag{18}$$

The upper update uses a zero budget for the nonnegative deficit: a violation increases the penalty up to its limit, while a satisfied requirement gives $d_w^R = 0$ and leaves the multiplier unchanged. The lower update is applied at trajectory end and can decrease the multiplier when the violation frequency falls below budget.

E. Training and Execution

Algorithm **??** gives the online scheduling procedure, and Algorithm **??** gives joint training. Each training trajectory spans a complete workload. After delayed costs are assigned to their originating decisions, transitions are inserted into replay and the upper reliability multiplier is updated once per completed workflow. The upper networks are trained first, followed by SAC; the remaining multipliers are then updated for subsequent training. Target networks use soft updates.

Replay transitions connect consecutive scheduling decisions, including decisions from different workflows. Only the last decision of a complete trajectory is terminal; minibatches are sampled uniformly. During SAC updates, rewards are recomputed from stored components using the current multipliers.

Online scheduling requires one forward pass through each upper network to evaluate all servers and one actor evaluation for the selected task–server pair. The critics and counterfactual replay are used only during training. For a trajectory containing $|\mathcal{W}_e|$ workflows, counterfactual feedback requires $|\mathcal{W}_e| + 1$ replay passes. Each pass visits every task and edge in the dependency-and-server-order graph once. The subsequent allocation considers $|\mathcal{W}_e|^2$ workflow pairs without reevaluating either policy.

During evaluation, network parameters and multipliers remain fixed. Server selection is greedy, and the frequency policy uses its deterministic output. Constraint attainment is assessed using the deadline and reliability criteria in Section **??**.

Algorithm 1 Online workflow scheduling with CAHDRL

Require: Server pool $\mathcal{M}$, trained Q_E, Q_D, Q_R and π_L, and stored multipliers.
Ensure: Task assignments, frequency ratios, and workflow outcomes.

Initialize $\tau_{\text{now}} = 0$, $t = 0$, availability $A_m = 0$ for all m, and empty server queues.
while arrivals remain or any workflow is unfinished **do**
  if no task is ready for assignment **then**
    Advance τ_{now} to the next arrival or completion; reveal arriving workflows, update finished tasks, server queues, and ready tasks, and record outcomes of newly completed workflows.
  else
    Select the ready task i with the smallest workload L_i; let w be its workflow.
    Form $\boldsymbol{o}_t^H$ and choose m_i using (**??**).
    Form $\boldsymbol{o}_t^L$ for (i, m_i), obtain ρ_i^a from the deterministic policy, and apply (**??**) to obtain ρ_i.
    Compute $ST(i)$ and $FT(i)$ via (**??**)–(**??**); enqueue i on m_i at $ST(i)$ with ratio ρ_i, set $A_{m_i} = FT(i)$, and schedule its completion.
    Remove i from the ready tasks and record its energy and reliability; set $t \leftarrow t + 1$.
  end if
end while
return Task assignments, frequency ratios, and workflow outcomes.

Algorithm 2 Joint training of CAHDRL

Initialize both agents, target networks, replay buffers, and controllers.
for each training trajectory **do**
  Collect a complete trajectory with Algorithm **??**, using ε-greedy server selection and stochastic frequency samples; retain transitions, task times, and immediate feedback.
  Compute workflow outcomes and upper deadline/reliability costs; replay the fixed schedule with all workflows, then each workflow separately restored to full frequency; allocate lower deadline costs via (**??**)–(**??**).
  Complete decision transitions and insert them into replay; update μ_R^H per completed workflow using (**??**).
  Update the three upper networks and the SAC critics and actor using the learning rules in Section **??**, followed by soft target updates.
  Update μ_D^H, μ_D^L, and μ_R^L when their windows and warm-up conditions permit.
end for

V. Experimental Evaluation

The evaluation examines energy at measured constraint attainment, component effects, cross-source transfer, and sensitivity to workload size and arrival pressure. All results concern fixed checkpoints in the common simulator.

TABLE II
WORKLOAD GROUP SAMPLING PROBABILITIES.

Workload scale	Small group (24–30 tasks)	Medium group (47–58 tasks)	Large group (97–100 tasks)
Small	1.00	0.00	0.00
Medium	0.30	0.70	0.00
Large	0.20	0.30	0.50

TABLE III
WORKLOAD–RESOURCE SCENARIOS.

Scenario	Workload scale	Edge	Cloud	Total
SS	Small	28	9	37
SM	Small	56	18	74
SL	Small	84	27	111
MS	Medium	28	9	37
MM	Medium	56	18	74
ML	Medium	84	27	111
LS	Large	28	9	37
LM	Large	56	18	74
LL	Large	84	27	111

A. Experimental Setup

1) Workloads and Resources: Workflow topologies are drawn from CyberShake, Epigenomics, LIGO/Inspiral, Montage, and SIPHT [**?**], with size distributions given in Table **??**. Templates are sampled uniformly within each size group. Task workloads and dependency data volumes are independently sampled from $[0.5, 20] \times 10^9$ CPU cycles and $[1, 30]$ MB, respectively.

Table **??** combines three workload scales with three resource configurations; the first and second scenario letters indicate workload and resource scale. Server profiles follow the power and fault models in [**?**], [**?**], with simulation parameters listed in Table **??**. Profiles are approximately equally represented within each layer. For all servers, the minimum frequency ratio is fixed at $\underline{\rho}_m = 0.1$, corresponding to $F_m^{\min} = 0.1F_m$, and the fault-rate sensitivity is set to $\kappa_m = 3$. Maximum frequencies and active-power components are independently perturbed by up to $\pm 5\%$ using seed 42; all methods share the resulting server pools. Edge–edge, edge–cloud, and cloud–cloud bandwidths are 1, 0.1, and 10 Gbit/s, respectively. We set $R_w = 0.90$. Ready tasks are dispatched in increasing workload order.

2) Deadlines and Evaluation Protocol: Following Cai et al. [**?**], deadlines are generated as

$$T_w^{\mathrm{d}} = a_w + \alpha_w M_w^{\mathrm{ref}}, \qquad \alpha_w \in \{2, 3\}.$$

The sampling probabilities over $\{2, 3\}$ are $(0.8, 0.2)$, $(0.5, 0.5)$, and $(0.2, 0.8)$ for Tight, Medium, and Loose deadlines, respectively. Reference makespans use isolated full-frequency HEFT schedules during training and validation, and shared-load, full-frequency FCFS/first-idle schedules during evaluation. Evaluation references account for communication and contention and are recomputed per resource configuration. All methods receive identical test workloads and deadlines.

TABLE IV
RESOURCE SETTING

Profile	F_m (GHz)	P_m^{ind} (W)	P_m^{dyn} (W)	$\lambda_m^{\max}$ (s^{-1})
Edge low-end	1.80	3.00	12.80	10^{-5}
Edge efficient	2.40	5.87	36.03	10^{-5}
Edge legacy	2.90	18.06	102.34	10^{-5}
Cloud standard	3.25	29.16	213.84	6×10^{-6}
Cloud fast	3.60	41.32	371.88	6×10^{-6}

Models are trained separately for each resource configuration on Small workflows with Medium deadlines, 20 workflows per realization, and exponential interarrival times at rate $0.1\ \mathrm{s}^{-1}$. Main tests use 50 workflows at $0.03\ \mathrm{s}^{-1}$ and 20 paired realizations per condition, yielding $9 \times 3 \times 4 \times 20 = 2160$ runs. At each resource scale, one checkpoint is tested across all workload scales and deadline regimes with frozen networks and multipliers. Repetitions vary workloads, not training initializations.

3) Compared Methods and Training Settings: Three published methods are adapted to the common simulator, task/resource features, and dispatch rule.

- *DQN-Edge* [**?**] retains separate time/energy Q-estimators and context-aware attention, with full-frequency execution. We replace its task ordering and reward streams with common dispatch, incremental deadline costs, and placement energy savings; constraint penalties enter the time stream, and attention targets follow the adapted rewards.
- *DReaM-DQN* [**?**] retains joint placement and seven-level frequency selection, replacing cores with servers and using an MLP with simulator-specific features and rewards. Training uses simulator interactions; fault-tolerance actions, redundant execution, and deployment-time learning are omitted.
- *OOAC* [**?**] retains binary-coded actions, actor candidate generation, critic screening, and binary-cross-entropy actor updates, with 100 candidates, $s = 30$, and five frequency levels. Training uses bit perturbations; evaluation generates deterministic candidates ranked by Hamming distance and actor likelihood, then scores finalists using one-step estimates without future-arrival information.

Discrete frequencies span $[\underline{\rho}_m, 1]$. OOAC uses simulator rewards without the original wireless fading, routing, or transmission-energy models. Comparisons concern these adapted implementations. DQN-Edge/DReaM-DQN use energy weights $3/1$, deadline weights $10/10$, and reliability weights $1/10$. OOAC uses a placement deadline weight of 10 and a DVFS reward weight of 0.1, with deadline and reliability weights of 10 within that component. Test sets are not used for tuning.

CAHDRL's upper/lower agents use hidden-layer widths $(256, 256)/(128, 128)$, Adam learning rates $10^{-4}/(3 \times 10^{-4})$, batch sizes $256/128$, and $5/10$ updates per trajectory after sufficient replay is available. Buffer capacity is 50,000 per agent, the target-update coefficient is 0.005, and the gradient-norm limit is 1. We set $\delta_E = \delta_D = \delta_R = \delta_L = 0.99$. Upper

exploration decreases from 1 to 0.05 by 5×10^{-4} per update. SAC uses entropy coefficient 0.2, $\rho_0 = 0.2$, and log-standard-deviation bounds $[-5, 2]$.

Reward settings are $\xi_h = \zeta = 0.05$, $\epsilon_T = 10^{-12}$ s, $\beta_{\text{miss}} = 1$, and $\theta_q = 0.20$. Energy/delay standard-deviation floors are $0.05/0.005$, with standardized components clipped to $[-3, 3]$. Calibrating β_q over 50 training trajectories gives 5.605, 5.208, and 3.573 for 37, 74, and 111 servers, respectively. Training projections use a fixed per-task reliability estimate of 0.999 for unscheduled tasks.

Deadline-controller settings are $(b_H, b_L) = (0.10, 0.08)$, $(\tau_H, \tau_L) = (0.10, 0.20)$, $(k_H, k_L) = (3, 25)$, and $(\bar{\mu}_D^H, \bar{\mu}_D^L) = (6, 8)$. Upper/lower warm-ups are $50/20$ trajectories and windows are $60/40$ trajectories; the lower statistic averages the largest 25% of costs. Reliability settings are $(\tau_R^H, \tau_R^L) = (10^{-3}, 0.5)$, $(\bar{\mu}_R^H, \bar{\mu}_R^L) = (100, 50)$, and $b_R = 0.005$. The upper budget is zero; the lower window contains 100 workflows without additional warm-up. All multipliers and PI accumulators start at 1.

All models are trained for 1200 episodes, with validation every 20 or 50 episodes on two disjoint validation blocks. Feasible checkpoints are ranked by computation energy, and no test outcome is used in their selection. For DQN-Edge, the smoothed attention state is carried across test realizations, which are evaluated in the recorded order.

4) Evaluation Metrics: For $N_{\text{test}} = 20$ realizations, we report:

- *Computation energy (EC)*: total active computation energy per realization in J, reported as the mean and, where indicated, sample standard deviation.
- *Empirical success probability (ESP)*:

$$\text{ESP}_\Gamma = \frac{100\%}{N_{\text{test}}} \sum_{e=1}^{N_{\text{test}}} \mathbb{I}(\text{SR}_e \geq \Gamma).$$

  ESP@90 uses $\Gamma = 0.90$ and is reported as a percentage.
- *Reliability attainment (Rel.)*:

$$\text{Rel.} = \frac{100\%}{N_{\text{test}}} \sum_{e=1}^{N_{\text{test}}} \mathbb{I}(\mathcal{R}_w \geq R_w,\ \forall w \in \mathcal{W}_e)\,.$$

  This measures attainment of reliability requirements, not fault-free execution.

With $\eta = 0.90$, empirical feasibility requires ESP@90 $\geq 90\%$ and Rel. $= 100\%$. Bold values mark the lowest mean energy satisfying both criteria. Both attainment metrics have five-percentage-point resolution.

B. Energy Performance

Table **??** and Fig. **??** show that CAHDRL has the lowest mean computation energy in all 27 conditions, with reductions of 10.31–57.26% relative to the lowest-energy adapted baseline in each condition. The largest reduction occurs in SS–Loose (201024.39 versus 470354.39 J for DReaM-DQN); the smallest occurs in LL–Tight (998546.81 versus 1113308.48 J for OOAC). CAHDRL's energy increases from Loose to Tight at every workload/resource scale, consistent with the cost of meeting tighter deadlines.

C. Deadline and Reliability Performance

CAHDRL, DQN-Edge, and OOAC attain 100% ESP@90 and reliability attainment in all 27 conditions; DReaM-DQN attains 65–100% and 35–100%, respectively. Thus, CAHDRL's energy advantage is retained without a loss of measured attainment. ESP@90 of 100% means that every tested batch meets the 90% on-time target, not that every workflow is on time. Likewise, Rel. of 100% denotes satisfaction of the modeled reliability threshold, not fault-free execution or a population-level guarantee.

D. Ablation and Diagnostic Comparisons

Table **??** presents the component ablation in SM (74 servers, 50 Small workflows, arrival rate $0.03\ \text{s}^{-1}$). Each deadline regime uses 20 paired realizations (seeds 742000–742019), disjoint from the main-experiment seeds; EC is reported as mean $\pm$ sample standard deviation, and FF denotes full-frequency execution.

- *CAHDRL*: the complete hierarchy, using the main-experiment checkpoint.
- *D3QN-FF*: the same assignment policy with all tasks at full frequency.
- *D3QN-RFR*: the same assignment policy with SAC replaced by the reference ratio $\widehat{\rho}_{im}$.
- *HEFT-SAC*: queue-aware HEFT assignment with a frequency agent trained under it.
- *HEFT-FF*: the same HEFT scheduler at full frequency.
- *D3QN-ND-FF*: a full-frequency assignment policy trained without the deadline head and multiplier, retaining the energy and reliability heads.

CAHDRL, D3QN-FF, and D3QN-RFR share the same assignment networks and multipliers, so the latter two are evaluation-time interventions rather than retrained policies. HEFT-SAC and D3QN-ND-FF are trained separately; the latter has no feasible validation checkpoint and is retained as a failure diagnostic. All checkpoints are fixed across regimes, and every variant receives identical paired workloads and deadlines.

1) Task Assignment: D3QN-FF consumes less energy than HEFT-FF while both satisfy the sample criteria, showing that the energy advantage is not solely due to DVFS. HEFT-FF is unchanged across regimes because its assignments and frequencies are deadline-independent. HEFT-SAC reduces energy but attains only 0–5% reliability, failing the joint criteria in all regimes.

2) Frequency Scaling: Enabling SAC reduces energy relative to D3QN-FF by 4.54%, 4.57%, and 3.87% under Loose, Medium, and Tight deadlines, with both attainment metrics at 100%. This intervention measures the effect of frequency control with fixed network weights, not a separately trained assignment-only policy. Placements need not remain identical because changed execution times alter subsequent queues and observations.

D3QN-RFR reduces energy further but attains 0% reliability in every regime, despite ESP@90 values of 100%, 100%, and 90%. A local timing budget and an energy-efficient floor therefore do not suffice to preserve workflow reliability with this assignment policy. The comparison supports learned

TABLE V
ENERGY CONSUMPTION AND CONSTRAINT SATISFACTION ACROSS RESOURCE SCALES AND DEADLINES.

		DQN-Edge			DReaM-DQN			OOAC			CAHDRL		
DDL	Scenario	EC (J)	ESP	Rel.	EC (J)	ESP	Rel.	EC (J)	ESP	Rel.	EC (J)	ESP	Rel.
Loose	SS	1304446.16	100%	100%	470354.39	100%	100%	583538.17	100%	100%	**201024.39**	100%	100%
	SM	483148.78	100%	100%	404754.43	100%	100%	520216.36	100%	100%	**333124.63**	100%	100%
	SL	394513.97	100%	100%	419794.85	100%	100%	470950.64	100%	100%	**329561.70**	100%	100%
	MS	1871333.01	100%	100%	695590.48	100%	100%	827903.22	100%	100%	**341954.83**	100%	100%
	MM	852671.72	100%	100%	677294.71	100%	95%	805030.12	100%	100%	**556511.43**	100%	100%
	ML	837399.26	100%	100%	662284.98	100%	100%	722113.87	100%	100%	**552999.05**	100%	100%
	LS	2710163.42	100%	100%	1077658.70	100%	60%	1182715.47	100%	100%	**604508.95**	100%	100%
	LM	1512637.01	100%	100%	1160554.58	95%	90%	1189993.94	100%	100%	**945580.94**	100%	100%
	LL	1822503.74	100%	100%	1064915.16	100%	100%	1082617.50	100%	100%	**928041.09**	100%	100%
Medium	SS	1320884.70	100%	100%	470178.45	100%	100%	581063.59	100%	100%	**214481.33**	100%	100%
	SM	610090.66	100%	100%	443233.28	100%	100%	527958.60	100%	100%	**352734.70**	100%	100%
	SL	483258.98	100%	100%	444712.55	100%	100%	480377.56	100%	100%	**343854.16**	100%	100%
	MS	1893791.51	100%	100%	696661.37	95%	100%	825742.68	100%	100%	**364249.41**	100%	100%
	MM	992463.98	100%	100%	733300.39	100%	100%	816278.84	100%	100%	**580957.60**	100%	100%
	ML	996348.44	100%	100%	697476.90	100%	100%	733678.00	100%	100%	**573241.34**	100%	100%
	LS	2718545.11	100%	100%	1086443.12	90%	55%	1188403.26	100%	100%	**636746.42**	100%	100%
	LM	1649760.23	100%	100%	1230625.39	95%	95%	1211070.96	100%	100%	**982835.90**	100%	100%
	LL	2047663.43	100%	100%	1119191.92	100%	100%	1098158.36	100%	100%	**965714.53**	100%	100%
Tight	SS	1336890.82	100%	100%	466884.28	100%	100%	573476.97	100%	100%	**227250.92**	100%	100%
	SM	718681.72	100%	100%	476644.33	100%	100%	537863.92	100%	100%	**371409.34**	100%	100%
	SL	562309.90	100%	100%	467293.15	100%	100%	491064.30	100%	100%	**359454.13**	100%	100%
	MS	1911670.06	100%	100%	701952.03	100%	100%	816674.06	100%	100%	**382162.67**	100%	100%
	MM	1109288.72	100%	100%	779372.27	100%	100%	827674.26	100%	100%	**603189.51**	100%	100%
	ML	1132480.18	100%	100%	727977.67	100%	100%	744717.84	100%	100%	**593451.69**	100%	100%
	LS	2732409.68	100%	100%	1107135.79	65%	35%	1192872.27	100%	100%	**665041.56**	100%	100%
	LM	1795642.67	100%	100%	1283625.59	95%	95%	1228507.52	100%	100%	**1021983.90**	100%	100%
	LL	2231710.75	100%	100%	1161695.57	100%	100%	1113308.48	100%	100%	**998546.81**	100%	100%

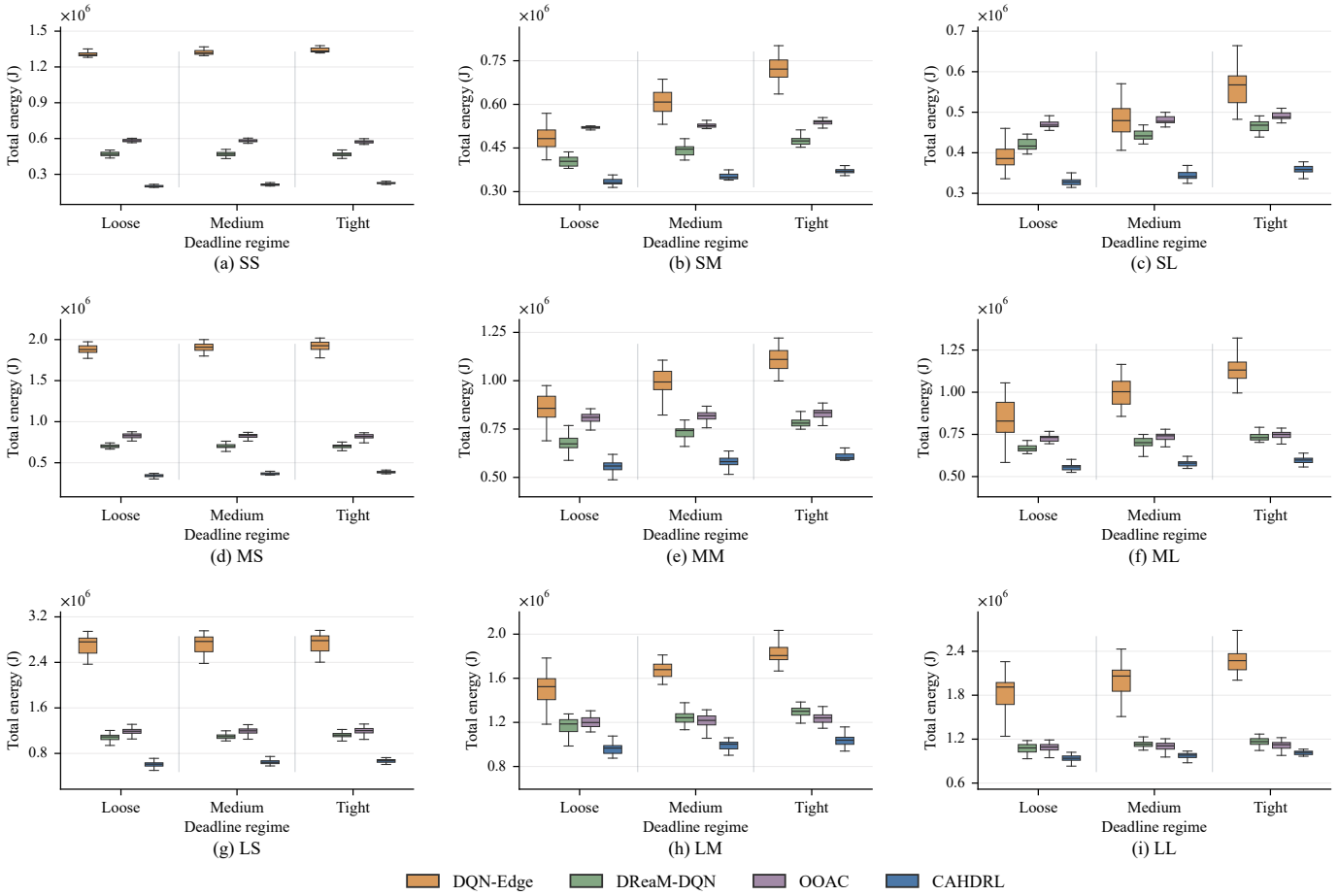


Fig. 2. Energy consumption distributions of the evaluated algorithms under varying resource scales and deadlines.

TABLE VI
ABLATION STUDY IN SM.

	Loose			Medium			Tight		
Variant	EC	ESP	Rel.	EC	ESP	Rel.	EC	ESP	Rel.
CAHDRL	$\mathbf{340860 \pm 13518}$	100%	100%	$\mathbf{358004 \pm 15724}$	100%	100%	$\mathbf{378346 \pm 14165}$	100%	100%
D3QN-FF	357054 ± 13280	100%	100%	375152 ± 15611	100%	100%	393583 ± 15858	100%	100%
D3QN-RFR	202268 ± 7855	100%	0%	214710 ± 9874	100%	0%	225233 ± 9132	90%	0%
HEFT-SAC	610270 ± 10787	100%	0%	619054 ± 11004	100%	0%	625596 ± 10625	100%	5%
HEFT-FF	1502920 ± 21142	100%	100%	1502920 ± 21142	100%	100%	1502920 ± 21142	100%	100%
D3QN-ND-FF	126463 ± 2378	0%	100%	126414 ± 2127	0%	100%	126624 ± 2413	0%	100%

frequency control over direct execution of RFR in the tested setting, rather than isolating the effect of the reliability penalty alone.

3) Deadline Feedback: D3QN-ND-FF achieves zero ESP@90 across all 60 realizations despite full reliability attainment. Its low energy is therefore infeasible. This diagnostic supports the need for deadline-aware assignment, but does not isolate the deadline head: D3QN-ND-FF is trained at full frequency, whereas CAHDRL is trained jointly with SAC.

Exact two-sided McNemar tests [**?**] compare D3QN-RFR with CAHDRL and D3QN-FF. Reliability differences remain significant after Holm correction [**?**] across 12 comparisons (three regimes, two metrics, two comparators; adjusted $p = 2.29 \times 10^{-5}$). Deadline differences are not detected (uncorrected $p = 1$, 1, and 0.5 for Loose, Medium, and Tight).

4) Reliability Diagnostic: A separate LM–Tight diagnostic uses the same 74-server checkpoint and seeds 742000–742019 (1,000 workflows). Minimum and first-percentile workflow reliabilities are 0.99406 and 0.99439, above the 0.90 requirement; the task-weighted mean frequency ratio is 0.97781. The complete policy thus retains substantial reliability margin in this condition. Reliability feedback was nevertheless active during training: 36 trajectories violated the requirement, and sampled SAC reliability penalties were nonzero in 132 trajectories. These observations establish feedback activation and retained margin, not the isolated causal benefit of the reliability penalty.

E. Cross-Source Evaluation on Alibaba Workflows

We test cross-source transfer without retraining using Alibaba cluster-trace-v2018 DAGs [**?**] and the frozen 111-server models. Preprocessing preserves dependencies, merges each task's instances, and infers workload intensity from duration, requested CPU, and instance count. A mapping calibrated on 40,000 jobs disjoint from the 10,000-job test set assigns 0.5–20 Gcycles and 1–30 MB; neither calibration nor checkpoint selection uses test outcomes. Each of 20 paired realizations samples 100 jobs without replacement on 84 edge and 27 cloud servers. Methods share jobs, arrivals, and FCFS-reference deadlines in each regime. Batches contain 794–1,125 tasks (mean 918.05), distinguishing job count from processing load.

CAHDRL consumes 128620 ± 13310, 135470 ± 15140, and 142150 ± 16690 J under Loose, Medium, and Tight deadlines (Fig. **??**). Reductions against the lowest-energy baseline—DQN-Edge for Loose and OOAC otherwise—are 24.25%, 27.16%, and 28.48%. All methods attain 100% on both constraints. These results support transfer to trace-derived DAGs under the stated simulation mapping, rather than validation on a deployed Alibaba system.

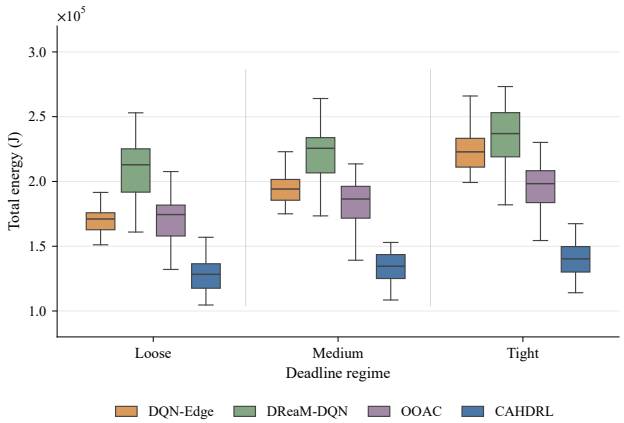


Fig. 3. Computation energy distributions on Alibaba-derived workflows under three deadline regimes.

F. Sensitivity to Workflow Count and Arrival Rate

Two SM–Tight tests separate cumulative workload size from arrival pressure, using frozen models and 20 paired realizations per setting. Bars and error bars in Fig. **??** denote means and sample standard deviations. The count test increases workflows from 50 to 250 in steps of 50 at $0.03\ \mathrm{s}^{-1}$, regenerating shared-load FCFS deadlines per batch. The arrival-rate test uses 50 workflows at 0.01–$0.20\ \mathrm{s}^{-1}$ and fixes relative deadline budgets to the $0.03\ \mathrm{s}^{-1}$ reference, so increased contention does not relax those budgets.

1) Workflow Count: CAHDRL's total energy rises approximately linearly from 378350 to 1871500 J, while energy per workflow remains near 7570–7490 J. Savings over DReaM-DQN, the lowest-energy baseline, remain 22.15–22.40%, and all methods attain 100% on both constraints. The advantage therefore persists over larger batches; at fixed arrival rate, this is not a test of increasing arrival pressure.

2) Arrival Rate: As arrival rate increases, CAHDRL's mean task waiting time rises from 1.00 to 2.21 s and energy from 372950 to 453440 J. It retains the lowest mean energy throughout, with a 9.02% advantage over OOAC at $0.20\ \mathrm{s}^{-1}$ despite the narrowing gap.

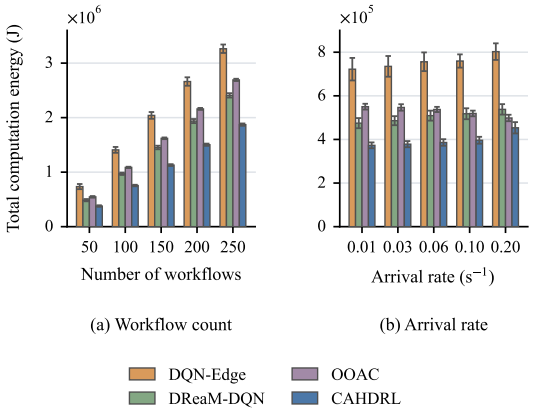


Fig. 4. Sensitivity of computation energy to (a) workflow count and (b) arrival rate.

All methods attain 100% ESP@90 at 0.01–0.10 s^{-1}. At 0.20 s^{-1}, CAHDRL and OOAC retain 100%, whereas DQN-Edge and DReaM-DQN attain 90%. Reliability attainment remains 100% throughout. Thus, CAHDRL preserves its energy advantage and measured attainment over the tested range; finite batches do not establish steady-state capacity.

G. Statistical Significance

Exact two-sided Wilcoxon tests [**?**] compare 20 paired energy observations per condition. Bonferroni and Benjamini–Hochberg corrections [**?**] cover 54 comparisons with DReaM-DQN and OOAC. The reported DQN-Edge runs retain attention state across realizations and are excluded from independent-pairs inference. CAHDRL consumes less energy in every tested pair, yielding identical exact p-values that remain significant after correction (Table **??**). The tests concern the reported workload blocks and fixed checkpoints, not training-seed variability or population feasibility.

TABLE VII
PAIRED WILCOXON TESTS FOR MAIN-EXPERIMENT COMPUTATION ENERGY: 27 TESTS PER BASELINE; — DENOTES EXCLUSION.

Baseline	p-value	Bonferroni	FDR-BH
DQN-Edge	—	—	—
DReaM-DQN	1.91×10^{-6}	1.03×10^{-4}	1.91×10^{-6}
OOAC	1.91×10^{-6}	1.03×10^{-4}	1.91×10^{-6}

H. Scope of the Evidence

The results establish energy savings and empirical constraint attainment for the evaluated policies. Each trainable variant uses one initialization; HEFT-SAC resumes without replay contents. Component interventions do not isolate every feedback term, and the results do not establish that counterfactual attribution is superior to simpler feedback. Reliability is evaluated analytically under independent task faults, while idle and communication energy are excluded. Training-repeat variability and measured online scheduling overhead remain outside this evaluation.

VI. Conclusion

CAHDRL coordinates discrete task assignment and continuous frequency control through layer-specific feedback and Lagrangian-inspired adaptive penalties. On the evaluated scientific-workflow blocks, it reduces mean computation energy by 10.31–57.26% relative to the lowest-energy adapted baseline while satisfying both empirical attainment criteria. Component comparisons distinguish placement and frequency-control effects, and trace-derived and load-sensitivity tests retain an energy advantage without retraining. These findings support the complete scheduling framework under the modeled conditions, rather than a feasibility guarantee or independent superiority of every feedback mechanism. Future work will examine training variability, time-varying networks, and online adaptation.

References

[1] A. Taghinezhad-Niar and J. Taheri, "Fault-tolerant cost-efficient scheduling for energy and deadline-constrained IoT workflows in edge-cloud continuum," *IEEE Trans. Services Comput.*, vol. 18, no. 5, pp. 2892–2903, 2025.

[2] R. Siyadatzadeh, F. Mehrafrooz, M. Ansari, B. Safaei, M. Shafique, J. Henkel, and A. Ejlali, "ReLIEF: A reinforcement-learning-based real-time task assignment strategy in emerging fault-tolerant fog computing," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10 752–10 763, 2023.

[3] D. Zhu, R. Melhem, and D. Mossé, "The effects of energy management on reliability in real-time embedded systems," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design*, 2004, pp. 35–40.

[4] M. Esnaashari, M. Ansari, and A. Ejlali, "DReaM: Deep reinforcement learning for joint reliability and power management in multicore IoT devices," *IEEE Internet Things J.*, vol. 12, no. 24, pp. 54 509–54 522, 2025.

[5] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, no. 1, 2016.

[6] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn.*, ser. Proc. Mach. Learn. Res., vol. 48, 2016, pp. 1995–2003.

[7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. 35th Int. Conf. Mach. Learn.*, ser. Proc. Mach. Learn. Res., vol. 80, 2018, pp. 1861–1870.

[8] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, 2002.

[9] V. Arabnejad, K. Bubendorfer, and B. C. K. Ng, "Dynamic multi-workflow scheduling: A deadline and cost-aware approach for commercial clouds," *Future Gener. Comput. Syst.*, vol. 100, pp. 98–108, 2019.

[10] Z. Wang, W. Zhan, H. Duan, G. Min, and H. Huang, "Deep-reinforcement-learning-based continuous workflows scheduling in heterogeneous environments," *IEEE Internet Things J.*, vol. 12, no. 10, pp. 14 036–14 050, 2025.

[11] A. Jayanetti, S. Halgamuge, and R. Buyya, "Deep reinforcement learning for energy and time optimized scheduling of precedence-constrained tasks in edge–cloud computing environments," *Future Gener. Comput. Syst.*, vol. 137, pp. 14–30, 2022.

[12] G. P. Koslovski, K. Pereira, and P. R. Albuquerque, "DAG-based workflows scheduling using actor–critic deep reinforcement learning," *Future Gener. Comput. Syst.*, vol. 150, pp. 354–363, 2024.

[13] M. Wen, X. Liu, X. Ning, C. Liu, X. Chen, J. Nian, and L. Cheng, "Deep reinforcement learning for energy-efficient workflow scheduling in edge computing," *Comput. Netw.*, vol. 274, p. 111790, 2026.

[14] M. Wen, X. Ning, M. Pu, C. Liu, Q. Wang, X. Chen, and L. Cheng, "Energy-efficient real-time workflow scheduling in edge–cloud environments based on multi-agent reinforcement learning," *Appl. Energy*, vol. 415, p. 127866, 2026.

[15] B. Xiao, C. Yu, X. Chen, Z. Chen, and G. Min, "Multi-agent collaboration for workflow task offloading in end-edge-cloud environments using deep reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 36, no. 11, pp. 2281–2296, 2025.

[16] X. Song, J. Feng, L. Liu, Q. Pei, F. R. Yu, and N. Zhang, "A deep reinforcement learning with transformer integration for directed acyclic graph scheduling in edge networks," *IEEE Trans. Wireless Commun.*, vol. 25, pp. 5506–5520, 2026.

[17] L. Zhang, K. Li, C. Li, and K. Li, "Bi-objective workflow scheduling of the energy consumption and reliability in heterogeneous computing systems," *Inf. Sci.*, vol. 379, pp. 241–256, 2017.

[18] G. Xie, G. Zeng, X. Xiao, R. Li, and K. Li, "Energy-efficient scheduling algorithms for real-time parallel applications on heterogeneous distributed embedded systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 12, pp. 3426–3442, Dec. 2017.

[19] E. Oustad, A. Younesi, M. Ansari, S. Safari, M. A. Soleimani, J. Henkel, and A. Ejlali, "DIST: Distributed learning-based energy-efficient and reliable task scheduling and resource allocation in fog computing," *IEEE Trans. Services Comput.*, vol. 18, no. 3, pp. 1336–1351, 2025.

[20] L. Ye, L. Yang, X. Zhao, and Y. Xia, "An energy-aware multistages hybrid scheduling approach for IoT workflow applications with reliability constraint in cloud computing systems," *IEEE Internet Things J.*, vol. 12, no. 15, pp. 29 670–29 681, 2025.

[21] ——, "Energy-minimized scheduling for reliable workflow applications in heterogeneous cloud computing systems," *Inf. Sci.*, vol. 730, p. 122856, Mar. 2026.

[22] Z. Li, S. Ren, and G. Quan, "Energy minimization for reliability-guaranteed real-time applications using DVFS and checkpointing techniques," *J. Syst. Archit.*, vol. 61, no. 2, pp. 71–81, 2015.

[23] H. A. Hassan, S. A. Salem, and E. M. Saad, "A smart energy and reliability aware scheduling algorithm for workflow execution in DVFS-enabled cloud environment," *Future Gener. Comput. Syst.*, vol. 112, pp. 431–448, 2020.

[24] B. Hu, Z. Cao, and M. C. Zhou, "Energy-minimized scheduling of real-time parallel workflows on heterogeneous distributed computing systems," *IEEE Trans. Services Comput.*, vol. 15, no. 5, pp. 2766–2779, 2022.

[25] Z. Wu, L. Han, J. Liu, Y. Robert, and F. Vivien, "Energy-aware mapping and scheduling strategies for real-time workflows under reliability constraints," *J. Parallel Distrib. Comput.*, vol. 176, pp. 1–16, 2023.

[26] A. Kouloumpris, G. L. Stavrinides, M. K. Michael, and T. Theocharides, "A reliability- and latency-driven task allocation framework for workflow applications in the edge-hub-cloud continuum," *Future Gener. Comput. Syst.*, vol. 180, p. 108414, 2026.

[27] W. Masson, P. Ranchod, and G. Konidaris, "Reinforcement learning with parameterized actions," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, 2016, pp. 1934–1940.

[28] L. Ale, S. A. King, N. Zhang, A. R. Sattar, and J. Skandaraniyam, "D3PG: Dirichlet DDPG for task partitioning and offloading with constrained hybrid action space in mobile-edge computing," *IEEE Internet Things J.*, vol. 9, no. 19, pp. 19 260–19 272, 2022.

[29] Y. Gu, H. Zhou, C. Liu, F. Fang, Z. Zhao, G. Theodoropoulo, and L. Cheng, "A hierarchical GNN-based multi-agent framework for workflow scheduling in hybrid clouds considering privacy constraints," *IEEE Trans. Services Comput.*, vol. 19, no. 3, pp. 1905–1918, 2026.

[30] P. K. S, L. M. Rao, and R. Ghogge, "Cost–reliability-aware multi-workflow scheduling using multi-agent deep reinforcement learning in edge–cloud systems," *Ingénierie des Systèmes d'Information*, vol. 31, no. 3, pp. 889–899, 2026.

[31] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proc. 34th Int. Conf. Mach. Learn.*, ser. Proc. Mach. Learn. Res., vol. 70, 2017, pp. 22–31.

[32] A. Stooke, J. Achiam, and P. Abbeel, "Responsive safety in reinforcement learning by PID lagrangian methods," in *Proc. 37th Int. Conf. Mach. Learn.*, ser. Proc. Mach. Learn. Res., vol. 119, 2020, pp. 9133–9143.

[33] L. Zhang, K. Li, Y. Xu, J. Mei, F. Zhang, and K. Li, "Maximizing reliability with energy conservation for parallel task scheduling in a heterogeneous cluster," *Inf. Sci.*, vol. 319, pp. 113–131, 2015.

[34] D. Zhu and H. Aydin, "Reliability-aware energy management for periodic real-time tasks," *IEEE Trans. Comput.*, vol. 58, no. 10, pp. 1382–1397, 2009.

[35] ——, "Energy management for real-time embedded systems with reliability requirements," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design*. IEEE, Nov. 2006, pp. 528–534.

[36] T. Wu, H. Gu, J. Zhou, T. Wei, X. Liu, and M. Chen, "Soft error-aware energy-efficient task scheduling for workflow applications in DVFS-enabled cloud," *J. Syst. Archit.*, vol. 84, pp. 12–27, 2018.

[37] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, "Characterizing and profiling scientific workflows," *Future Gener. Comput. Syst.*, vol. 29, no. 3, pp. 682–692, 2013.

[38] K. Cai, Q. Wu, M. Zhou, C. Chen, J. Wen, and S. Wang, "Dynamically scheduling deadline-constrained interleaved workflows on heterogeneous computing systems," *IEEE Trans. Services Comput.*, vol. 18, no. 2, pp. 758–769, 2025.

[39] Y. Wang and S. Sun, "Dynamic task scheduling in wireless edge computing using deep reinforcement learning with ordinal optimization," *EURASIP J. Wireless Commun. Netw.*, vol. 2025, no. 1, p. 96, 2025.

[40] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[41] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[42] Alibaba, "Alibaba Cluster Trace Program: cluster-trace-v2018," https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018, accessed: Oct. 17, 2026.

[43] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.

[44] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *J. Roy. Statist. Soc. Ser. B (Methodological)*, vol. 57, no. 1, pp. 289–300, 1995.